

PROJECT AND TEAM INFORMATION

Project Title

Advanced Mathematical Expression Processing and Evaluation System

Student / Team Information

<i>Team Name:</i> <i>Team #</i>	<i>The Postfix Crew</i>
Team member 1 (Team Lead)	<i>Prajapati, Vanshika:</i> <i>2510380242</i> <i>2510380242@geu.ac.in</i> 
Team member 2	<i>Shagun – 2510380250</i> <i>2510380250@geu.ac.in</i> 
Team member 3	<i>Bisht, Jai –2510011540</i> <i>2510011540@geu.ac.in</i> 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Calculators and Software need a smart way to solve mathematical problems. While simple mathematics like $2+2$ is easy, complex formulas with brackets, symbols, and variables need to be organized carefully before a computer can provide the right answers. This project builds a C++ program that takes complex mathematical equations and organizes them in stacks queues for solving them efficiently. It uses a well-known rule set named the Shunting Yard Algorithm, which figures out which operation happens first. It converts regular mathematical expressions into formats that are easier for computers to understand. It puts equations into a Tree-Like structure to demonstrate how all parts are connected to each other. It uses techniques like Stack, Recursion, hashing, and searching to follow the map and provide the solution.

State of the Art / Current solution (1 pt)

Calculators and software hide how they solve the mathematical problems step by step. They just provide you the final solution rather than giving you the brief about how the solution has occurred.

This project builds a tool which clearly shows each and every step behind the solution. So users can learn how data structures and algorithms work while solving the advanced equations.

Therefore, the project combines calculation functionality with practical visualization of DSA concepts.

Project Goals and Milestones (2 pts)

The main goal of this project is to build a c++ programming tool using Key Concepts of DSA such as Stacks, Trees, Recursion, Hashing, Shunting yard algorithm which processes, convert and evaluate advance mathematical expression consisting of variables, mathematical functions and error handling.

Planning: *Analyzing the requirements and designing the overall architecture.*

Processing: *Implementation of tokenization, expression conversions (Prefix/Postfix/Infix), evaluation and expression trees.*

Handling: *Variable handling and expression simplifications.*

Final outcome: *Testing, optimizing, documentation and presentation.*

Project Approach (3 pts)

The project breakdown advance mathematical expression into subpart categories like numbers, symbols, variables and functions which are further checked for syntax errors like mismatched parentheses or symbol error.

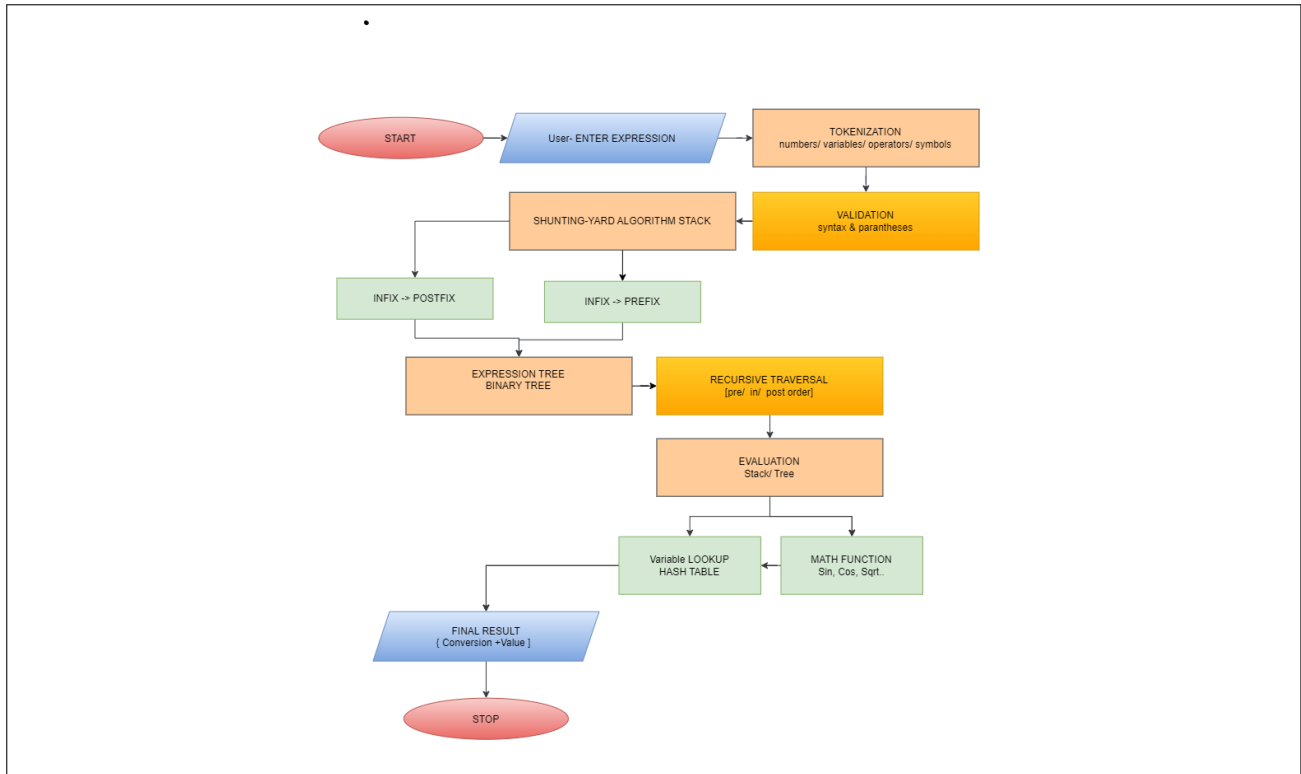
Then using Stacks and Shunting yard algorithm for handling the precedence while switching expressions between infix, postfix and prefix formats.

Building a binary expression tree with postfix expressions and tree traversing (pre-order, post-order, in-order) to determine all three mathematical notations.

Additionally using hash tables for lookup for variables. Built in C++ mathematical libraries for functions and exception handling to detect error (division by zero/ undefined variables)

System Architecture (High Level Diagram)(2 pts)

used Draw.io .



Project Outcome / Deliverables (1 pts)

The expected outcome is a functional C++ mathematical expression processing and evaluation engine capable of accepting and processing mathematical expressions.

0.1 The system will provide:

*Infix, Prefix, and Postfix conversion.
Shunting-Yard implementation.
Prefix and Postfix evaluation.
Binary Expression Tree construction.
Recursive tree traversal.
Variable handling using hashing.
Expression validation and error handling.
Expression simplification.*

0.1.1 Deliverables

*Complete C++ source code.
Working project executable file.
System architecture and flowchart.
Project documentation/report.
Test cases and sample outputs.
Final presentation/PPT.*

Assumptions

The project assumes that user will provide a mathematical expression using supporting operators, numbers, functions and variables which will follow the standard mathematical rules. Then the system handles the basic numerical and simple variables expressions while assigning the values to the variables before solving the expression. Further more the program runs a c++ command line application in the initial phase.

References

***Object Oriented Programming with C++**, McGraw Hill.*
*Paul Deitel and Harvey Deitel, **C++: How to Program**, Pearson.*
*Mark Allen Weiss, **Data Structures and Algorithm Analysis in C++**.*
***GeeksforGeeks** — Data Structures and Algorithms Programiz — C++ Programming*